Session 25 Homework — Redux Current User, Resolved Names, and Workspace/Member Management

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context (required)

By the end of class, you have a real, working login/register flow, a real httpOnly session cookie, and a real protected route group. One real gap remains, disclosed since Session 23: Issue Detail still shows raw user ids for reporter/assignee, because resolving them needs `GET /users/:id`, which genuinely requires auth this app never had until this session. You also have no way, anywhere in the app, to show WHO is currently signed in, or to log out. This homework closes both gaps (CORE), then transfers the generic Server-Action-form pattern already taught to two new real product features (STRETCH), then asks you to reason in writing about this session's own design (CHALLENGE).

Goal (required)

A real current-user snapshot, available to Client Components, that a navbar-style component can read without itself being able to `await` the backend (CORE); real names resolved on Issue Detail instead of raw ids (CORE); a real Create Workspace flow, and a real Workspace Settings page where an admin/owner can search for, add, promote/demote, and remove members (STRETCH); a real, independent written answer connecting this session's own httpOnly cookie, protected layout, and Redux snapshot into one coherent explanation (CHALLENGE).

What you already have

A real, working `loginAction` / `registerAction`, a real httpOnly session cookie (`requireSessionToken` / `apiFetch` with token+401 support), and a real protected `(app)` route group wrapping every existing page. `GET /users/:id` and `GET /users/search` both already exist and work once a real token is attached (confirm their real shapes yourself — don't guess them). The real backend's own workspace-member routes already exist too, the same way.

What to build (required)

A real Redux Toolkit slice holding exactly one thing (the current user or `null`); a real client/server hydration bridge so a Server Component's own already-fetched user reaches that slice; a real account-bar-style Client Component reading it, with a real logout action; a real `/account` page; Issue Detail resolving real names via the already-existing `GET /users/:id`; a real Create Workspace form/page; a real Workspace Settings page with search/add/role-change/ remove, reachable from both desktop and the mobile navigation.

CORE (required)

How to start

Install `@reduxjs/toolkit` and `react-redux` first. Build the slice itself before anything that uses it — one reducer, setting the whole user value at once, never field by field.

Hints

- **Hint 1 (conceptual):** a real Next.js server handles many real requests concurrently — think about what goes wrong if the function that creates your Redux store is called once at the module level, shared across every request, instead of once PER request.
- **Hint 2 (where to look):** the protected layout already calls `requireSessionToken()` — it is also the one place that can call a second real endpoint to fetch who that token belongs to, before anything renders.
- **Hint 3 (API or tool name):** `useRef` (for creating the store exactly once inside a Client Component), `Provider` (from `react-redux`).

What to research

1. Why does the Redux `Provider` component need to be inside a Client Component, when the current user's own data was fetched by a Server Component?
2. What is the real, structural reason `useSelector((s) => s.currentUser.user)` is written that way, rather than `useSelector((s) => s.currentUser)`?

STRETCH (required)

How to start

Build Create Workspace first — it is the smaller of the two features, and it is the identical real `useActionState` + `apiFetch` + inline-error pattern `registerAction` already used, applied to a different real backend endpoint.

Hints

- **Hint 1 (conceptual):** a real duplicate-slug attempt against `POST /workspaces` returns the same KIND of real backend response `registerAction`'s own duplicate-email case already taught you to handle — think about where you've already solved this exact shape of problem.
- **Hint 2 (where to look):** `GET /users/search` is called DIRECTLY from a Client Component, as a plain function call, not bound to a `<form>` — this is deliberately a different way to use a Server Action from everything shown live. Think about why a search-as-you-type box can't wait for a form submission.
- **Hint 3 (API or tool name):** the real backend's own workspace-member routes already support add/change-role/remove — confirm their real shapes (method, body, response) before writing any frontend code against them.

CHALLENGE (required)

How to start

Re-read this session's own three real mechanisms side by side: the httpOnly session cookie, the protected layout's own server-side check, and the Redux current-user snapshot. Write down, in one sentence each, what real question each one actually answers.

Hints

- **Hint 1 (conceptual):** only one of the three ever touches the real token directly. Which one, and why do the other two never need to?
- **Hint 2 (where to look):** think about what would ACTUALLY go wrong — concretely, not just "it would be less secure" — if any one of the three were removed while keeping the other two.

What to research

1. If `currentUserSlice` were somehow stale or wrong, could an unauthenticated request still get through to a protected page? Why or why not?
2. If the httpOnly flag were removed from the session cookie, what specifically would become possible that isn't possible today?

Rules (required)

Allowed: everything already shown LIVE/TEAMWORK this session (`"use server"`, `useActionState`, `apiFetch` with token support, `requireSessionToken`, Redux Toolkit's `createSlice` / `configureStore`).

Not yet: no Issue create/edit/delete, no drag-and-drop board, no Sprint/Backlog — all of that is Session 26. No `@mention` picker, no Activity feed, no Notifications UI — Session 27.

Test it (required)

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
npx tsc --noEmit
npm run build
```

Then, in a real browser: log in, confirm your own real name appears somewhere on every protected page, and that logging out actually clears your session; open an issue and confirm real names appear instead of raw ids; create a real new workspace; open its Settings, search for a real teammate, add them, change their role, then remove them.

Common confusions

- Treating `currentUserSlice` as the thing that performs authentication — it is populated only AFTER a real auth check already succeeded elsewhere.
- Two sequential `await`s instead of resolving reporter and assignee names together — works, but needlessly serializes two independent real network calls.
- Trusting a UI-side "is this the owner" check as the real security boundary for member management, instead of recognizing the real backend enforces it independently either way.

Done when (required)

- A real signed-in user's own name appears on every protected page, with a real way to log out.
- Issue Detail shows real names, not raw ids.
- A real new workspace can be created end to end, and becomes owned by its creator.
- A real admin/owner can search for, add, promote/demote, and remove a workspace member, from both desktop and the mobile navigation.
- Your written CHALLENGE answer addresses all three mechanisms (cookie, layout, Redux), not just one of them.
- `npx tsc --noEmit` and `npm run build` both exit clean.

Working time (required)

About 180 minutes total (HW-01…42) — CORE (HW-01…11) is ~68 minutes; STRETCH (HW-12…40, HW-42) is ~102 minutes; CHALLENGE (HW-41) is ~10 minutes.